

Why graphical models?

From observed alerts to a probability we can check

DSAA 4011 | Fall 2026–27

Preparation edition

Yuwen Huang | Week 1 lecture

Course outlook

The course studies how models combine local information to answer questions about an entire system.

Weeks 1–3	**Represent a model and compute exact probabilities.**
Weeks 4–8	Sample, learn, and approximate costly calculations.
Weeks 9–11	Learn computation rules that preserve useful structure.
Weeks 12–14	Study diffusion, attention, and language models.

Week 1 starts the foundation: define a structured probability model and compute an answer that can be checked exactly.

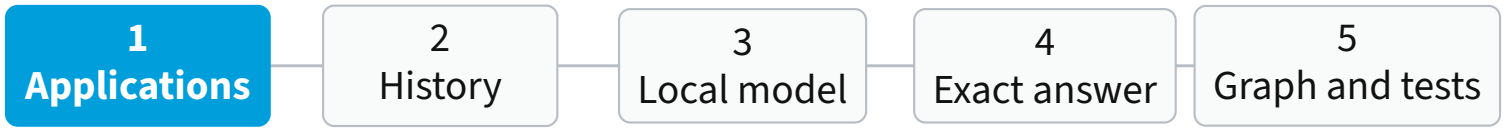
Week 1 overview

Start: infer a missing machine report.

Broaden: compare communication and magnetism.

Build: define variables and two local weight tables.

Finish: calculate a distribution, draw its factor graph, and test the result.



Current task: Week 1 overview.

Begin with a monitoring problem

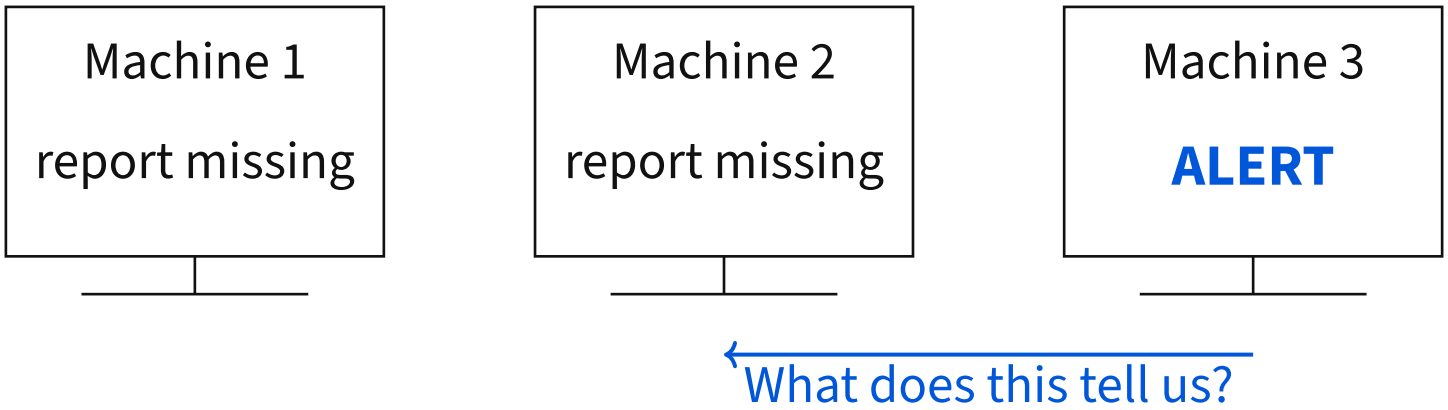
Question

Machine 3 reports an alert. What can we infer about Machine 2?

By the end

Separate observations, unknowns, assumptions, and conclusions.

Only one report is available



Machine 3 reports an **alert**. Reports from Machines 1 and 2 are missing.

Our question is about **Machine 2**: how likely is it to report an alert?

Missing and ``no alert'' differ

Report missing

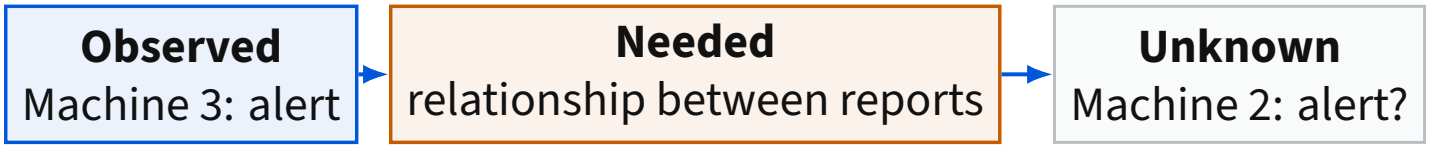
We have not observed that machine’s alert state.

No alert

We observed the report and its value is “no alert.”

Do not replace a missing report with “no alert.” Missing means unknown; “no alert” is an observed value.

One alert helps only if reports are related



Unrelated reports

Machine 3 supplies no information about Machine 2.

Shared power supply

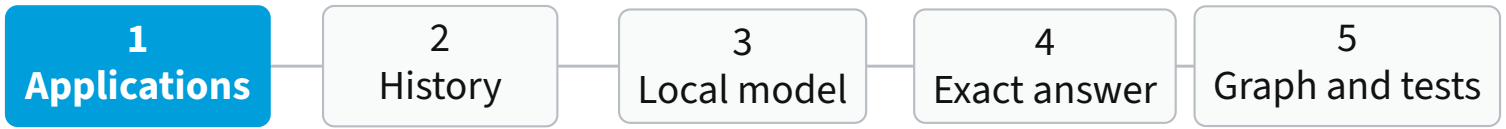
The two reports may occur together.

Key point: an observation becomes useful only through a stated relationship model.

One pattern links the applications

1. **Known:** What do we observe or specify?
2. **Model:** Which relationship connects the quantities?
3. **Goal:** What answer must we calculate?

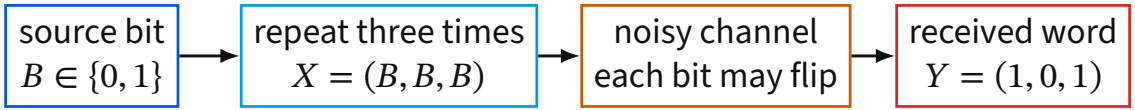
Monitoring: observed alert + stated relationship ⟶ missing-report probability.



Current task: One pattern links the applications.

Communication: define the system

Purpose: recover one transmitted bit after noise changes some received bits.



X_i: bit sent in position i; the receiver does not observe it.

Y_i: bit received in position i; the receiver observes it.

Probability: $P(\mathcal{A})$ denotes the probability of event $\mathcal{A}$.
Source: $B \in \{0,1\}$, with $P(B=0) = P(B=1) = \frac{1}{2}$.
Sent word: $X = (B,B,B)$.
Observed word: $Y = (1,0,1)$.

Question: was the source bit more likely to be $B=0$ or $B=1$?

The channel describes bit flips



$$P(Y_i = X_i) = 1 - \varepsilon$$

$$P(Y_i \neq X_i) = \varepsilon$$

$0 < \varepsilon < \frac{1}{2}$: bit-flip probability

The channel model states how a sent bit produces a received bit.

The channel acts independently at each position. Its bit-flip probability is ε.

For the observed value y_i, define the **channel likelihood**

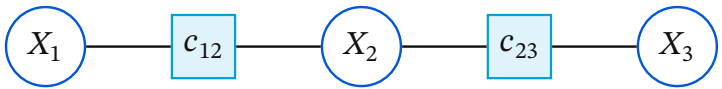
$$\ell_i(x_i) = P(Y_i = y_i \mid X_i = x_i).$$

It measures how well a proposed sent bit x_i explains the observed bit y_i.

The encoder imposes a constraint

Repetition encoding permits only 000 and 111. Define the **equality constraint**

$$c_{ij}(x_i, x_j) = \begin{cases} 1, & x_i = x_j, \\ 0, & x_i \neq x_j. \end{cases}$$



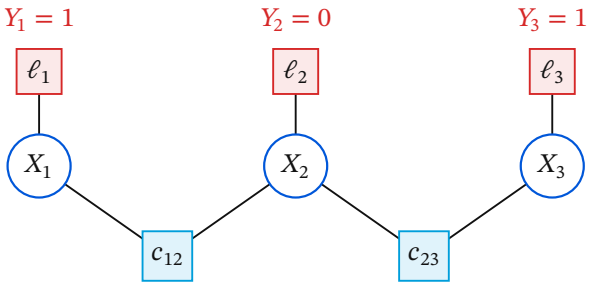
$c_{ij} = 1$ if $x_i = x_j$

$c_{ij} = 0$ otherwise

Both checks equal one only for 000 and 111.

The constraints describe which transmitted words are valid. They do not describe channel noise.

The graph combines both model parts



$\ell_i(X_i)$:
channel likelihood

$c_{ij}(X_i, X_j)$:
code constraint

edge:
variable is an input

Circles: unknown sent bits.
Red squares: channel likelihoods fixed by received bits.
Cyan squares: repetition-code constraints.

Factor graph: each edge links a function to one of its inputs.
An edge alone does not state causation.

Why multiply the channel terms?

Model assumption: given $X = x$, bit flips at different positions are independent.

Therefore, the probability of receiving the complete word $Y = y$ is

$$P(Y = y \mid X = x) = \ell_1(x_1)\,\ell_2(x_2)\,\ell_3(x_3).$$

Each $\ell_i(x_i)$: support supplied by received bit y_i for proposed sent bit x_i.
Constraint product $c_{12}c_{23}$: 1 for an allowed repetition word; 0 otherwise.

The product is justified by the channel-independence and code assumptions.

Score each valid transmitted word

For a candidate word $x = (x_1, x_2, x_3)$, define

$$s(x) = \ell_1(x_1)\ell_2(x_2)\ell_3(x_3)c_{12}(x_1, x_2)c_{23}(x_2, x_3).$$

Suppose $\varepsilon = 0.1$ and $Y = (1, 0, 1)$.

$$s(000) = 0.1 \times 0.9 \times 0.1 = 0.009, \qquad s(111) = 0.9 \times 0.1 \times 0.9 = 0.081.$$

Likelihood ratio: $s(111)/s(000) = 0.081/0.009 = 9$. The received word 101 is nine times more probable when 111 was sent.

Normalize the two explanations

Equal source prior: $P(B = 0) = P(B = 1) = 0.5$.

$$\text{posterior weights} = (0.5 \times 0.009,\ 0.5 \times 0.081) \propto (0.009,\ 0.081).$$

Normalize: divide both weights by $0.009 + 0.081 = 0.09$.

$$\big(P(B = 0 \mid Y = 101), P(B = 1 \mid Y = 101)\big) = (0.1, 0.9).$$

Meaning: under this source, code, and channel model, 111 has posterior probability 0.9.

From communication to physics

Communication used two local ingredients: channel likelihoods and code constraints.

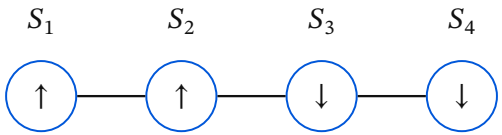
A physical model uses a different local ingredient: an interaction between neighboring states.

Next question: how can several neighboring interactions determine which complete state is likely?

Physics: define the system first

Purpose: show how local interactions favor some complete arrangements.

At site i, the binary variable $S_i \in \{-1, +1\}$ records whether its magnetic direction points down or up. The model assumes that only neighboring sites interact directly.



$S_i \in \{-1, +1\}$

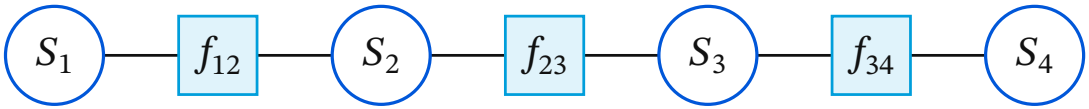
$+1$: arrow up

-1: arrow down

The model compares complete arrangements through neighboring interactions.

Question: which complete arrangements receive the largest values from the global function?

Physics: combine neighboring interactions



$f_{ij}(s_i, s_j) = 2$ when the two arrows align; otherwise $f_{ij}(s_i, s_j) = 1$.

$g(s_1, s_2, s_3, s_4) = f_{12}\, f_{23}\, f_{34}$ is the global function.

Square f_{ij}: local interaction function for neighboring sites i and j.
Aligned pair: weight 2. **Opposed pair:** weight 1.

Teaching values: 2 and 1 are model parameters, not measured probabilities.
Complete arrangement: multiply its three neighboring-interaction weights.

Why multiply interaction weights?

Local energy: $e_{ij}(s_i, s_j)$ scores neighboring directions s_i and s_j.
Total energy: $E(s) = e_{12} + e_{23} + e_{34}$.

For temperature $T > 0$, define

$$f_{ij}(s_i, s_j) = \exp\big(-e_{ij}(s_i, s_j)/T\big).$$

Then

$$\exp(-E(s)/T) = \exp(-e_{12}/T)\exp(-e_{23}/T)\exp(-e_{34}/T) = f_{12}f_{23}f_{34}.$$

Local energies add; their exponential weights multiply.

Physics: compare two arrangements

Global function: $g(s)$ multiplies the three neighboring-interaction values for arrangement s.
All four states align:

$$g(+1,+1,+1,+1) = 2 \times 2 \times 2 = 8.$$

Each neighboring pair is opposed:

$$g(+1,-1,+1,-1) = 1 \times 1 \times 1 = 1.$$

Comparison: the aligned arrangement has eight times the global-function value.
Next step: normalize all 16 values of g to obtain probabilities.

Normalize all sixteen arrangements

State space: four binary directions give $2^4 = 16$ arrangements.

$$\textit{Total}: \quad Z = \sum_s g(s) = 54, \qquad \textit{Probability}: \quad p(s) = \frac{g(s)}{54}.$$

All aligned

$p(+1, +1, +1, +1) = 8/54.$

Alternating

$p(+1, -1, +1, -1) = 1/54.$

Result: the 8-to-1 comparison remains, and the 16 probabilities sum to 1.

Each application uses a local model

Application	Small relationship used by the model
Monitoring	Alert states of connected machines.
Communication	Sent and received bits; neighboring code bits.
Magnetism	Directions at neighboring sites.

Local = limited scope: each relationship involves only a few quantities.

Local structure can shrink the model

Example: 20 binary variables, so each variable has two possible values.

One joint table

One entry per complete state.

$2^{20} = 1{,}048{,}576$ entries.

Markov-chain tables

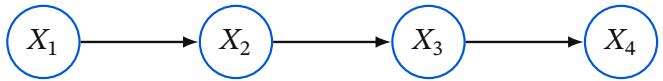
Initial table + 19 local transition tables.

$2 + 19 \times 4 = 78$ entries.

The saving is valid only when the Markov dependence assumption fits the problem.

A Markov chain states a dependence rule

Variable: $X_i \in \{0, 1\}$ is the state at time i.
Value: x_i is one possible value of X_i.



node: binary state

arrow: next-state dependence

$$P(x_1, x_2, x_3, x_4) = P(x_1)P(x_2 \mid x_1)P(x_3 \mid x_2)P(x_4 \mid x_3).$$

Assumption: after X_{i-1} is known, earlier states do not change the distribution of X_i.

Circle: random variable. **Arrow:** the preceding variable indexes the next-state table.

Local dependence rule ⟶ factorized probability.

Each transition has a conditional table

For the arrow $X_{i-1} \to X_i$, store the following table:

	$X_i = 0$	$X_i = 1$
$X_{i-1} = 0$	$P(X_i = 0 \mid X_{i-1} = 0)$	$P(X_i = 1 \mid X_{i-1} = 0)$
$X_{i-1} = 1$	$P(X_i = 0 \mid X_{i-1} = 1)$	$P(X_i = 1 \mid X_{i-1} = 1)$

Row: previous state. **Column:** next state. **Check:** each row sums to 1.

The transition table belongs to the arrow into X_i, not to the circle.

Two operations serve different purposes

Opening the box—understand the model.
Replace one global function by smaller local functions. The graph then exposes the model's conditional structure and stored tables.

Closing the box—compute more efficiently.
Evaluate a chosen subgraph over its internal variables. Store the result as one function of its boundary variables, and reuse that function in the remaining calculation.

Exactness requirement: both views must return the same value for every assignment of the variables left outside the box.

One function can hide the full model

Define the global function

$$g : \{0,1\}^4 \longrightarrow [0,1], \qquad g(x_1, x_2, x_3, x_4) = P(X_1 = x_1, \dots, X_4 = x_4).$$

What g contains: one probability for each of the $2^4 = 16$ complete state sequences.

What this closed view hides: which smaller conditional tables produced those 16 probabilities.

Define one local function per transition

Initial-state function:

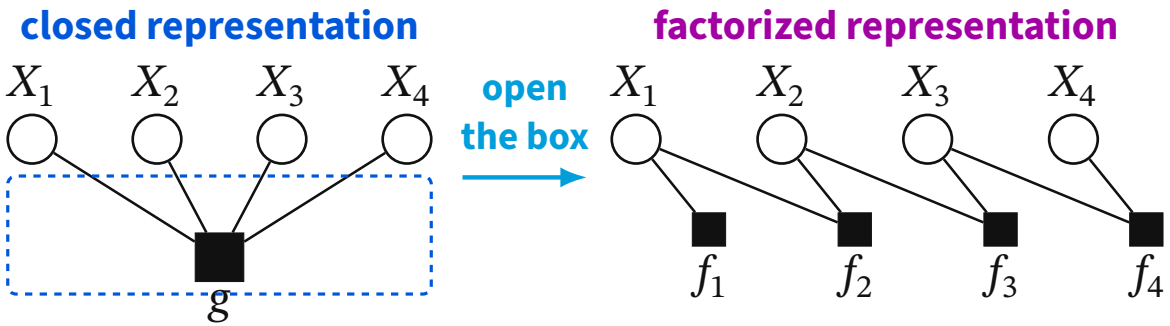
$$f_1(x_1) = P(X_1 = x_1).$$

Transition function: for $i \in \{2, 3, 4\}$,

$$f_i(x_{i-1}, x_i) = P(X_i = x_i \mid X_{i-1} = x_{i-1}).$$

Each directed transition becomes one two-variable local function.

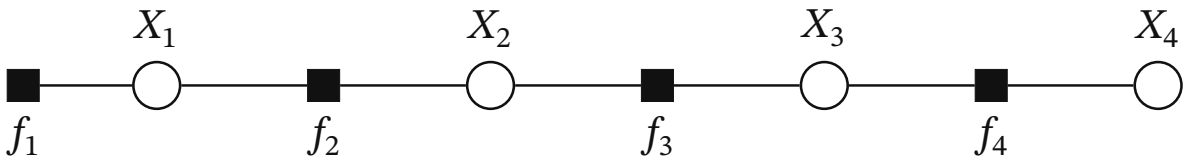
Opening the box reveals local structure



Your turn

The graph exposes f_1, f_2, f_3, f_4. Use the chain rule and the Markov assumption to predict how these functions reconstruct g.

The opened graph stores the same function



open circle: random variable filled square: local probability function

Your turn

Write the product represented by the graph. Check that every variable used by each function appears in that function's arguments.

Directed arrow: states the Markov conditional dependence.
Factor-graph edge: states only that a variable is an input to a local function.

Where the number 78 comes from

Without a dependence assumption: a joint table for 20 binary variables contains $2^{20} = 1{,}048{,}576$ entries—one for every complete assignment.

With the first-order Markov assumption:

Store $P(X_1)$: 2 entries. Then store one four-entry conditional table $P(X_i \mid X_{i-1})$ for each of the remaining 19 states.

$$\underbrace{2}_{P(X_1)} + \underbrace{19 \times 4}_{\text{transition tables}} = 78 \text{ stored probabilities.}$$

Because each table row sums to one, the model has $1 + 19 \times 2 = 39$ free parameters.

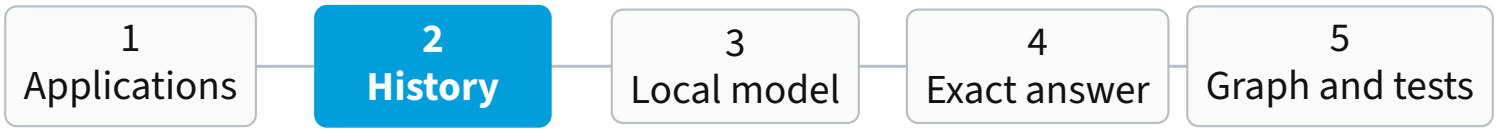
The reduction is justified only when the Markov assumption is suitable for the problem.

Where these ideas came from

Evidence + model ⟶ answer.

Small relationships ⟶ less storage and repeated work.

Historical links: Bayes | Markov | Shannon | Pearl.



Current task: Where these ideas came from.

History: update beliefs and limit dependence

1763 — Bayes

Problem: update a probability after observing evidence.
Course example: fault probability after an alert.

1906 — Markov

Problem: predict the next state without storing the full past.
Course example: tomorrow’s weather from today’s weather.

History: use constraints and visible structure

1948 — Shannon

Problem: recover a message corrupted by noise.

Course example: compare valid codewords with received bits.

1980s — Pearl

Problem: make dependence assumptions visible.

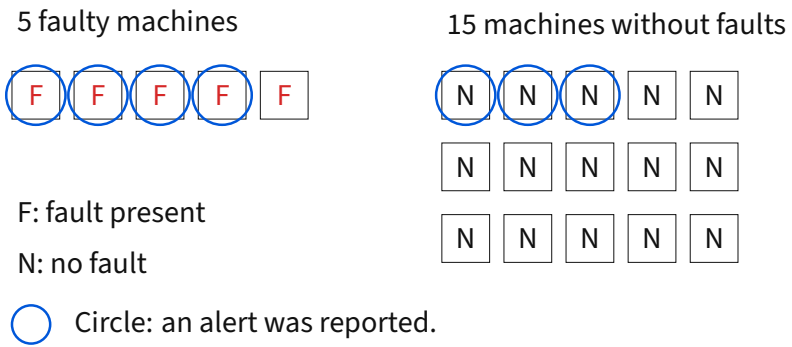
Course example: draw variables and their local relationships.

Shared lesson: write the structure before calculating the answer.

An alert can be a false alarm

Purpose: distinguish “probability of an alert given a fault” from “probability of a fault given an alert.”

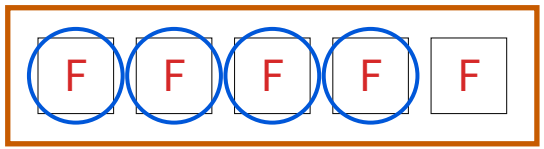
This separate example uses **20 machines**; the counts are invented for teaching. A **false alarm** is an alert reported by a machine that has no fault.



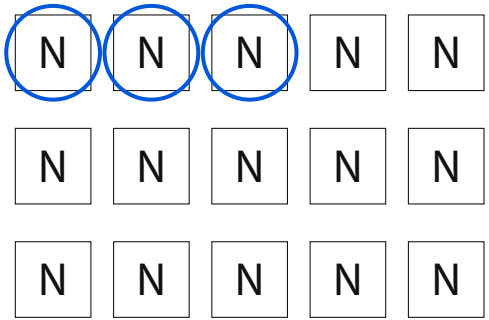
First select the faulty machines

Worked example

5 faulty machines



15 machines without faults



F: fault present

N: no fault

Circle: an alert was reported.

Among the 5 faulty machines, 4 report an alert. The requested fraction is $4/5$.

Now form the complete alert group

Alert: a machine reports an alert.

False alarm: an alert from a machine without a fault.

Alert group: 4 alerts with faults + 3 false alarms.

Your turn

Among machines that report an alert, what fraction are faulty? Draw the selected group.

Now form the complete alert group

Alert: a machine reports an alert.

False alarm: an alert from a machine without a fault.

Alert group: 4 alerts with faults + 3 false alarms.

Your turn

Among machines that report an alert, what fraction are faulty? Draw the selected group.

Hint: Count the complete alert group before forming the fraction.

The condition selects the denominator

Question 1: among faulty machines

Select 5 machines; 4 of them report an alert. The fraction is $4/5$.

Question 2: among machines with alerts

Select 7 machines; 4 of them are faulty. The fraction is $4/7$.

The wording after **among** chooses the comparison group and therefore the denominator.

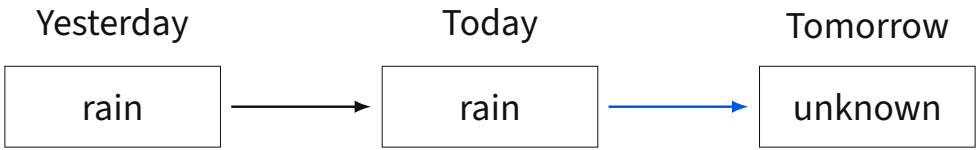
Different conditions, different answers:

$P(\text{alert} \mid \text{fault}) = 4/5; \qquad P(\text{fault} \mid \text{alert}) = 4/7.$

An alert does not prove that a fault exists.

Markov: state a local-dependence assumption

Purpose: show that a dependence statement is a modeling assumption that must be written explicitly.



Use today's state to predict tomorrow.

Model assumption: once today's weather is known, earlier weather adds no information about tomorrow.

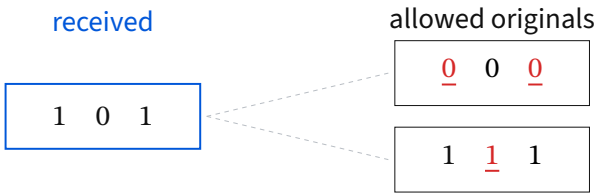
Takeaway: the assumption may simplify calculation, but data or domain knowledge must justify whether it is reasonable.

Shannon: use a hard noise limit

Purpose: revisit the repetition code with a simpler rule: at most one transmitted bit may change.

Worked example

Repeat 0 or 1 three times; assume **at most one symbol changes**.



Underlined symbols differ from the received string. Which allowed original, 000 or 111, is compatible with the assumption?

The hard limit selects one explanation

The received symbols are 101. The allowed originals are 000 and 111.

Compare with 000: two symbols differ, so 000 violates the “at most one change” rule.

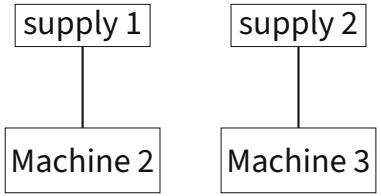
Compare with 111: one symbol differs, so 111 satisfies the rule.

Takeaway: the model’s local constraints rule out incompatible explanations. Later, local functions will encode such rules.

One situation can support two models

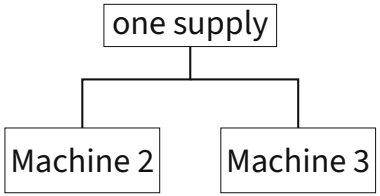
Purpose: show that different relationship assumptions can give different answers from the same observation.

A: independent power faults



Assume unrelated alerts.

B: a shared power fault



Alerts may occur together.

Separate supplies: independent-alert model.
Shared supply: dependent-alert model.
Choose the relationship model before calculating.

Model A follows from independence

Let $\mathcal{A}_2$ mean “Machine 2 reports an alert” and $\mathcal{A}_3$ mean “Machine 3 reports an alert.” Model A states

$$P(\mathcal{A}_2) = P(\mathcal{A}_3) = 0.1$$

and assumes that the two alert events are independent. Therefore,

$$P(\mathcal{A}_2 \cap \mathcal{A}_3) = P(\mathcal{A}_2)P(\mathcal{A}_3) = 0.1 \times 0.1 = 0.01.$$

Meaning: only 1% of monitoring periods contain both alerts under Model A.

Model B adds a joint-alert probability

Model B keeps $P(\mathcal{A}_2) = P(\mathcal{A}_3) = 0.1$ but specifies

$$P(\mathcal{A}_2 \cap \mathcal{A}_3) = 0.08.$$

The value 0.08 is an additional modeling assumption. It does not follow from the two values 0.1. In practice, data or knowledge of the shared supply would determine it.

	Machine 3: no alert	Machine 3: alert
Machine 2: no alert	0.88	0.02
Machine 2: alert	0.02	**0.08**

Check: $0.02 + 0.08 = 0.10$ for each machine. The two models have the same individual alert rates but different dependence.

Exact arithmetic answers the chosen model

We observe $\mathcal{A}_3$. Use $P(\mathcal{A}_2 \mid \mathcal{A}_3) = P(\mathcal{A}_2 \cap \mathcal{A}_3)/P(\mathcal{A}_3)$. Model A specifies joint probability 0.01; Model B specifies 0.08. Both specify $P(\mathcal{A}_3) = 0.1$.

Your turn

Compute $P(\mathcal{A}_2 \mid \mathcal{A}_3)$ under each model. If the shared-supply model describes the machines, can exact arithmetic under Model A recover the Model B answer?

Exact arithmetic answers the chosen model

We observe $\mathcal{A}_3$. Use $P(\mathcal{A}_2 \mid \mathcal{A}_3) = P(\mathcal{A}_2 \cap \mathcal{A}_3)/P(\mathcal{A}_3)$. Model A specifies joint probability 0.01; Model B specifies 0.08. Both specify $P(\mathcal{A}_3) = 0.1$.

Your turn

Compute $P(\mathcal{A}_2 \mid \mathcal{A}_3)$ under each model. If the shared-supply model describes the machines, can exact arithmetic under Model A recover the Model B answer?

Hint: Divide each joint alert probability by the same observed-alert probability, 0.1.

Return to the three-machine problem

Textual model	Each machine has two alert states. One local relationship links Machines 1 and 2; another links Machines 2 and 3.
Observed	Machine 3 reports an alert. Reports from Machines 1 and 2 are missing.
Question	What probabilities should the model assign to no alert and alert for Machine 2?

Next, define the uncertain quantities and local functions.
Then combine their weights and calculate the requested distribution.

Build the machine model

Return: three machines and one observed alert.

Define: variables, values, and two local weight tables.

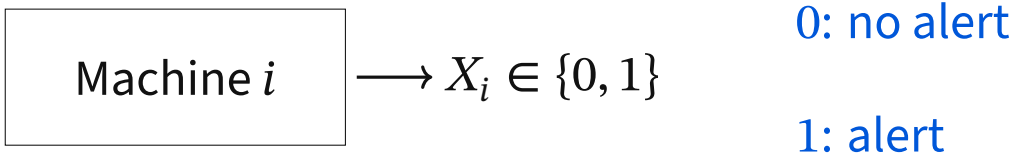
Deliverable: one numerical model ready for calculation.



Current task: Build the machine model.

Each variable has a stated meaning

A **random variable** is a named quantity whose value is uncertain before observation. For each index $i \in \{1, 2, 3\}$, define X_i as **Machine i's alert state**.

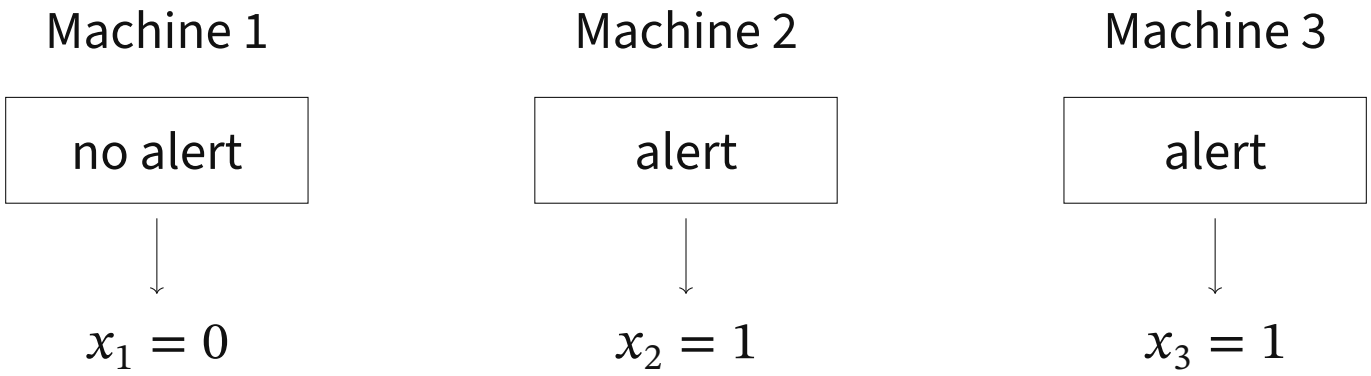


Its domain is $\mathcal{X}_i = \{0, 1\}$. A lowercase $x_i \in \mathcal{X}_i$ denotes one possible value.

An assignment specifies every variable

Worked example

A **complete assignment** supplies one value for each variable.



The ordered tuple $x = (x_1, x_2, x_3) = (0, 1, 1)$ says: no alert from Machine 1; alerts from Machines 2 and 3.

Read a second assignment

Consider the assignment $x = (1, 1, 0)$. The position of each entry identifies its machine.

Your turn

Which machines report alerts? If a fourth binary variable were added, how many complete assignments would exist?

Read a second assignment

Consider the assignment $x = (1, 1, 0)$. The position of each entry identifies its machine.

Your turn

Which machines report alerts? If a fourth binary variable were added, how many complete assignments would exist?

Hint: Read the tuple by position; each added binary variable doubles the assignment count.

State the target without new notation

Observation: $X_3 = 1$. Machine 3 reports an alert.

Target: two numbers for Machine 2, listed as

$$(\text{probability of no alert},\ \text{probability of alert}).$$

We will introduce formal probability notation only after defining the complete model.

Two binary machines have four pairs

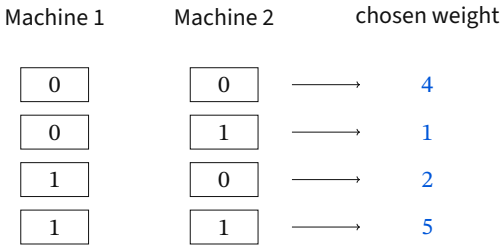
Machine 1	Machine 2
0	0
0	1
1	0
1	1

Each row is one value of $(x_1, x_2) \in \mathcal{X}_1 \times \mathcal{X}_2$.

A local function will assign a nonnegative weight to each pair.

Where do the local weights come from?

Purpose: assign a number to every pair of local machine states so that the model can compare the pairs.

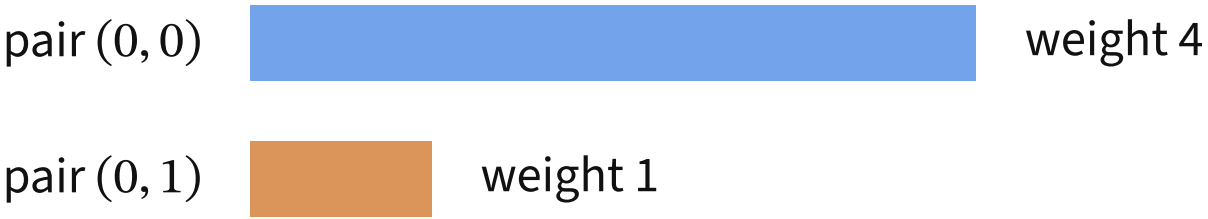


This week: the instructor supplies synthetic weights so that everyone can check the same inference calculation.

In an application: domain knowledge may specify the weights, or data may estimate them. Week 5 studies parameter learning.

A weight is a local relative contribution

Compare two local pairs while every other local function stays fixed.



Weight 4 contributes four times as much as weight 1 to the unnormalized model.

Not yet a probability: a local weight may exceed 1; local weights need not sum to 1.

Probability comes later: combine the local functions, then normalize.

A table stores four function values

Name the local function f_{12} because it uses Machines 1 and 2.

f_{12}			
	x_2	0	1
x_1	0	4	1
	1	2	5

Rows: the first input x_1.

Columns: the second input x_2.

The selected cell is the weight for that ordered pair.

Define a local function

A **local function** assigns a nonnegative weight to specified variable values. Define

$$f_{12} : \mathcal{X}_1 \times \mathcal{X}_2 \longrightarrow \mathbb{R}_{\geq 0}.$$

Its inputs are (x_1, x_2); its output is one nonnegative real number.

The four table entries define this function. A local function **need not sum to one**.

A second local function

Use the same rule for Machines 2 and 3. Define

$$f_{23} : \mathcal{X}_2 \times \mathcal{X}_3 \longrightarrow \mathbb{R}_{\geq 0}.$$

f_{12}

x_1 \ x_2	0	1
0	4	1
1	2	5

f_{23}

x_2 \ x_3	0	1
0	3	1
1	1	4

Machine 2 is an input to both local functions. Read each table in its declared argument order.

Read a table entry in two steps

Worked example

f_{12}

x_1 \ x_2	0	1
0	4	1
1	2	5

The first input $x_1 = 0$ selects **row 0**.

The second input $x_2 = 0$ selects **column 0**.

Therefore $f_{12}(0, 0) = 4$.

Read another table entry

In $f_{12}(x_1, x_2)$, the first input selects a row and the second selects a column.

Your turn

Find $f_{12}(1, 1)$. Why is the result a weight rather than a probability?

Read another table entry

In $f_{12}(x_1, x_2)$, the first input selects a row and the second selects a column.

Your turn

Find $f_{12}(1, 1)$. Why is the result a weight rather than a probability?

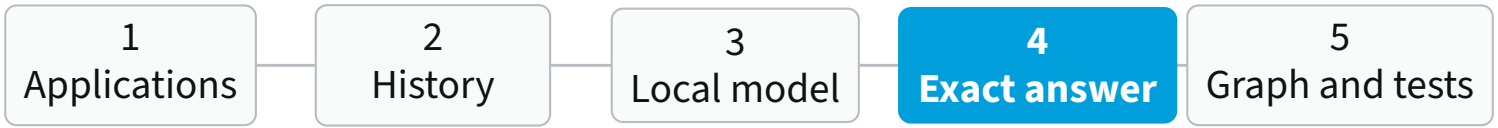
Hint: Use row 1 and column 1; then compare the result with the range of a probability.

Combine the local weights

Multiply: give each complete state one weight.

Sum: obtain the required normalizing total.

Answer: return both probabilities for Machine 2.



Current task: Combine the local weights.

Why multiply the two local weights?

A complete state $x = (x_1, x_2, x_3)$ receives one contribution from each local relationship. Define the **global function**

$$g(x) = \underbrace{f_{12}(x_1, x_2)}_{\text{Machines 1 and 2}} \times \underbrace{f_{23}(x_2, x_3)}_{\text{Machines 2 and 3}} .$$

Definition: $g : \mathcal{X} \to \mathbb{R}_{\geq 0}$ assigns one nonnegative value to each complete state.
Meaning: each local function contributes its support for the same complete state.
Modeling choice: multiplication is part of this model's definition; the graph alone does not impose it.
Useful consequence: if either local weight is zero, the complete state has zero weight.

The calculation must declare two operations

A **semiring** supplies two compatible operations: one combines local values; the other aggregates alternative states.

Calculation	Combine local values	Aggregate alternatives
Sum-product	multiply	add
Max-product	multiply	take the maximum
Min-sum	add	take the minimum

This week uses sum-product. Multiplication defines the global-function value for each complete state; addition produces totals and marginal probabilities. Other operations require a different declared meaning.

Aggregation depends on the semiring

After combining local values for each complete assignment, aggregate across the assignments:

Calculation	Aggregate over complete assignments
Sum-product	$Z = \sum_{x \in \mathcal{X}} g(x)$
Max-product	$M = \max_{x \in \mathcal{X}} g(x)$
Min-sum	$C = \min_{x \in \mathcal{X}} c(x)$

$c(x)$ denotes the total additive cost of assignment x. **Only sum-product uses Z to normalize probabilities.** Max-product and min-sum answer optimization questions instead.

Evaluate the global function once

Let $x = (x_1, x_2, x_3)$ denote a complete assignment. The global function is

$$g(x) = f_{12}(x_1, x_2) f_{23}(x_2, x_3).$$

Worked example

$$f_{12}(0,0) = 4 \qquad f_{23}(0,0) = 3$$

$$g(0,0,0) = 4 \times 3 = 12$$

Evaluate a second complete assignment

For $x = (1, 0, 1)$, use $x_1 = 1, x_2 = 0$ in f_{12} and $x_2 = 0, x_3 = 1$ in f_{23}.

Your turn

Locate both entries and compute $g(1, 0, 1)$.

Evaluate a second complete assignment

For $x = (1, 0, 1)$, use $x_1 = 1, x_2 = 0$ in f_{12} and $x_2 = 0, x_3 = 1$ in f_{23}.

Your turn

Locate both entries and compute $g(1, 0, 1)$.

Hint: Read $f_{12}(1, 0)$ and $f_{23}(0, 1)$ before multiplying.

Enumeration gives an exact reference

Your turn

Every row is one assignment $x \in \{0,1\}^3$. Multiply its two local-function values.

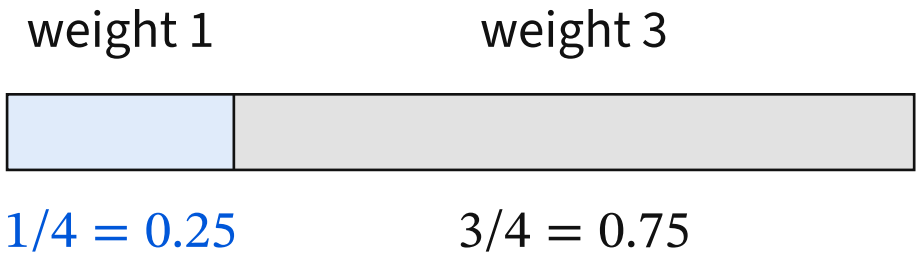
(x_1, x_2, x_3)	local-function entries	weight
000	4×3	12
001		
010		
011		
100		
101		
110		
111		

Complete all eight rows; then check that no assignment is missing or repeated.

Normalization converts relative weights

Worked example

For two outcomes with weights 1 and 3, first compute the total $1 + 3 = 4$.



The common total is $1 + 3 = 4$.

Divide each weight by the same positive total. The probabilities are $1/4$ and $3/4$; they are nonnegative and sum to one.

The partition function sums every state

Let $\mathcal{X} = \mathcal{X}_1 \times \mathcal{X}_2 \times \mathcal{X}_3 = \{0,1\}^3$ be the eight complete assignments. Using the enumeration table,

$$Z = \sum_{x \in \mathcal{X}} g(x) = 12 + 4 + 1 + 4 + 6 + 2 + 5 + 20 = ____.$$

Partition function: Z is the total weight of every complete state.

Compute Z and verify that it is positive before defining $p(x) = g(x)/Z$.

Check the normalized model

Use your total Z from the previous calculation. The same total normalizes every assignment.

Your turn

Find $p(1, 1, 1)$. State two checks on the resulting probability distribution.

Check the normalized model

Use your total Z from the previous calculation. The same total normalizes every assignment.

Your turn

Find $p(1, 1, 1)$. State two checks on the resulting probability distribution.

Hint: Divide the weight of state 111 by Z; then check nonnegativity and a total probability of one.

The joint distribution lists every state

For each complete state, use $p(x) = g(x)/54$.

x	$p(x)$	x	$p(x)$
000	_____	100	_____
001	_____	101	_____
010	_____	110	_____
011	_____	111	_____

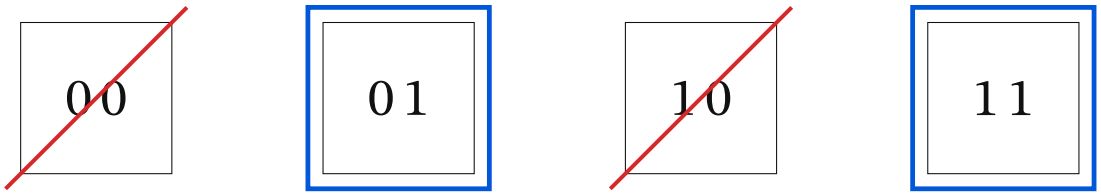
Your turn

Fill the table and check that the exact fractions sum to 1.

Conditioning retains compatible states

Worked example

Consider four equally likely cards: $00, 01, 10, 11$.



Observed last digit: 1. Retain 01 and 11.

One of the two retained cards begins with 1.

After observing that the last digit is 1, only 01 and 11 remain. Within that retained group, one of two cards begins with 1.

Define conditional probability

An **event** is a set of complete assignments. For events $\mathcal{A}$ and $\mathcal{B}$ with $p(\mathcal{B}) > 0$, define

$$p(\mathcal{A} \mid \mathcal{B}) = \frac{p(\mathcal{A} \cap \mathcal{B})}{p(\mathcal{B})}.$$

$\mathcal{B}$ is the observed event. The intersection $\mathcal{A} \cap \mathcal{B}$ contains assignments in both events.

Operationally: retain assignments in $\mathcal{B}$, then normalize within that group.

The observation selects four assignments

Your turn

The observed event is $\mathcal{B} = \{x \in \mathcal{X} : x_3 = 1\}$.

Each row of digits is (x_1, x_2, x_3).

0	0	0	0	0	1	0	1	0	0	1	1
1	0	0	1	0	1	1	1	0	1	1	1

Circle assignments ending in 1 and cross out the others.

Find the largest compatible weight

Given $X_3 = 1$, the compatible states are $001, 011, 101, 111$. Their global-function values come from $g(x) = f_{12}(x_1, x_2) f_{23}(x_2, x_3)$.

Your turn

Compute the four weights. Which complete state has the largest weight?

Find the largest compatible weight

Given $X_3 = 1$, the compatible states are $001, 011, 101, 111$. Their global-function values come from $g(x) = f_{12}(x_1, x_2) f_{23}(x_2, x_3)$.

Your turn

Compute the four weights. Which complete state has the largest weight?

Hint: Read both local-function entries for each state, multiply them, and compare the four products.

One best state is not a distribution

The previous calculation selects **one complete state**. Our original question asks for **both probabilities for Machine 2**:

$$\big(p(X_2 = 0 \mid X_3 = 1),\ p(X_2 = 1 \mid X_3 = 1)\big).$$

One state cannot supply both values. Group **every compatible state** by its value of X_2, add the weights in each group, and normalize the two totals.

Sum the compatible states for each answer

Let $j \in \{0, 1\}$ denote a possible value of X_2. Define its **unnormalized total**

$$u(j) = \sum_{x_1 \in \mathcal{X}_1} g(x_1, j, 1).$$

Your turn

Keep $x_2 = j$ and $x_3 = 1$ fixed. Sum over the two possible values of x_1.

Write before discussion.

Compatible states have total weight 30

Define

$$Z_{3=1} = u(0) + u(1).$$

Your turn

Compute $u(0)$ and $u(1)$, then add them to obtain $Z_{3=1}$.

All states: $Z = 54$ normalizes the eight-state joint distribution.

After observation: $Z_{3=1} = 30$ normalizes the four states compatible with $X_3 = 1$.

The conditional distribution has four states

For $x_1, x_2 \in \{0, 1\}$, define

$$p(x_1, x_2 \mid X_3 = 1) = \frac{g(x_1, x_2, 1)}{Z_{3=1}} = \frac{g(x_1, x_2, 1)}{30}.$$

	$x_2 = 0$	$x_2 = 1$
$x_1 = 0$	_____	_____
$x_1 = 1$	_____	_____

Your turn

Divide each compatible-state weight by 30 and check the total.

Sum over Machine 1 to answer

Machine 1 is unobserved and is not the target. Add down each column of the conditional table:

$$p(X_2 = j \mid X_3 = 1) = \sum_{x_1 \in \{0,1\}} p(x_1, j \mid X_3 = 1).$$

Your turn

Add the two entries in each column. State both probabilities.

Write before discussion.

The result answers the opening question

Your turn

Draw one bar representing probability 1. Divide it into the shares for no alert and alert.

Write before discussion.

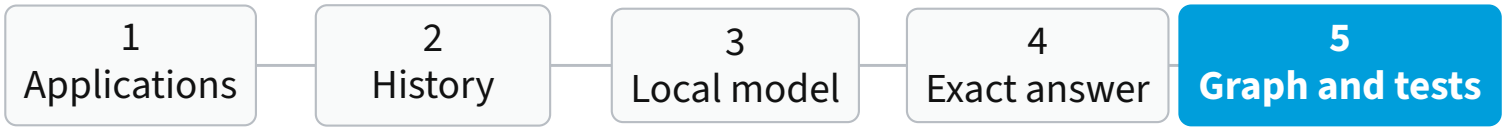
Label both shares and state the answer in one sentence.

Make the product structure visible

We used: three variables and two local functions.

We repeated: table lookup, multiplication, and summation.

Next: draw which variables enter each function.

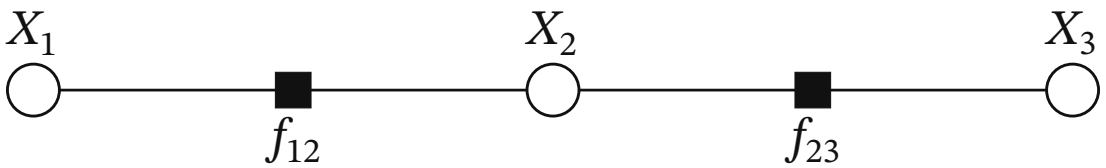


Current task: Make the product structure visible.

Function inputs define the graph

The global function has the following factorization:

$$g(x_1, x_2, x_3) = f_{12}(x_1, x_2) f_{23}(x_2, x_3).$$

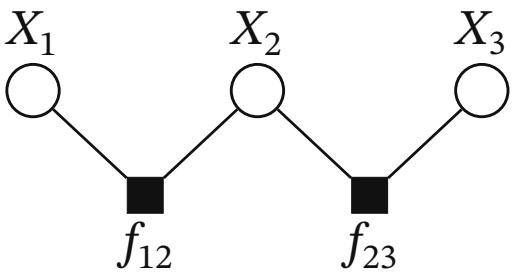


Each local function becomes a square. Its inputs become the neighboring variable circles. Thus, f_{12} connects to X_1, X_2, and f_{23} connects to X_2, X_3.

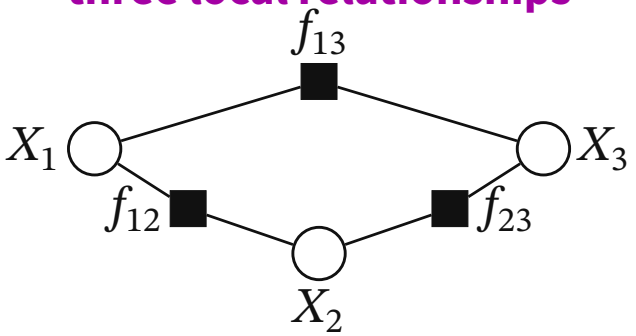
An extra function changes the topology

The same variables can support different factor graphs. The set of local functions determines which relationships the model includes.

two local relationships



three local relationships



Adding the local function $f_{13}(x_1, x_3)$ creates a direct modeled relationship between X_1 and X_3. It changes the state weights and, after normalization, may change the probability distribution. It does not by itself establish causation.

General notation: variables and values

$\mathcal{I}$	the set of variable indices;
X_i	the random variable with index $i \in \mathcal{I}$;
$\mathcal{X}_i$	the set of possible values of X_i;
$\mathcal{X} = \prod_{i \in \mathcal{I}} \mathcal{X}_i$	the set of all complete assignments;
$x = (x_i)_{i \in \mathcal{I}} \in \mathcal{X}$	one complete assignment, with each $x_i \in \mathcal{X}_i$.

General notation: function inputs

$\mathcal{F}$	the set of local functions;
$f \in \mathcal{F}$	one local function;
$\mathcal{I}_f \subseteq \mathcal{I}$	the indices of the variables used by f;
$x_f = (x_i)_{i \in \mathcal{I}_f}$	the input values supplied to f.

The number of inputs is not fixed. In the machine example, each local function happens to use two variables.

A local function returns one weight

The notation

$$f : \prod_{i \in \mathcal{I}_f} \mathcal{X}_i \longrightarrow \mathbb{R}_{\geq 0}$$

states the complete input and output specification.

Input: one value $x_i \in \mathcal{X}_i$ for every index $i \in \mathcal{I}_f$.

Output: one nonnegative real number $f(x_f)$, called a local weight.

This probability model uses sum--product

Your turn

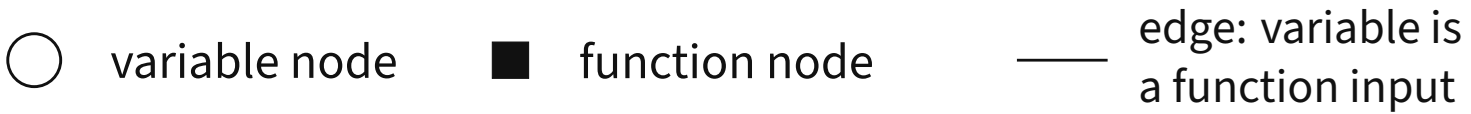
Complete the three steps: combine the local functions, total the complete states, and normalize.

$$g(x) = _____, \qquad Z = _____, \qquad p(x) = _____.$$

Your turn

Which two properties must be checked before calling p a probability distribution?

Translate function inputs into graph edges



Open circle: one variable.

Filled square: one local-function node.

Edge: the variable is an input to the local function.

Because $f_{12}(x_1, x_2)$ uses X_1 and X_2, connect f_{12} to those two variable nodes.

Build the graph from the functions

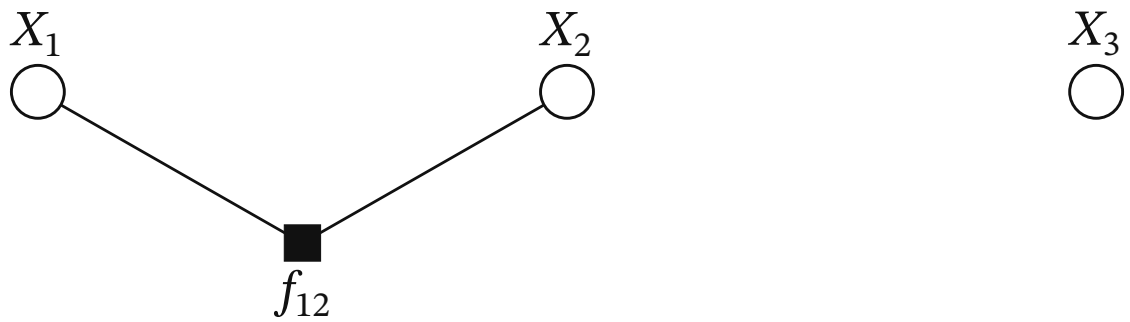
One variable, one circle. Add each function and its arguments.



Place the three variables. Their domains are $\{0, 1\}$.

Build the graph from the functions

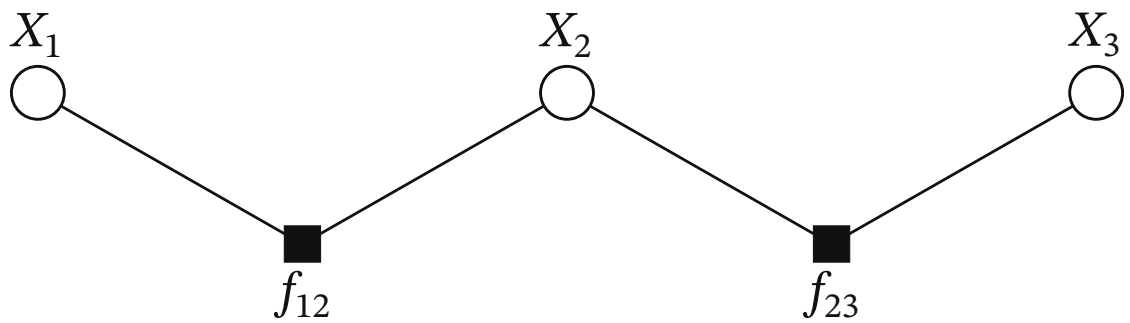
One variable, one circle. Add each function and its arguments.



The function f_{12} uses X_1 and X_2: draw exactly two edges.

Build the graph from the functions

One variable, one circle. Add each function and its arguments.



The function f_{23} uses X_2 and X_3. The shared variable has one node.

Define the factor graph precisely

A **factor graph** is a bipartite graph $G = (\mathcal{V}, \mathcal{F}, \mathcal{E})$ with two disjoint node sets:

$\mathcal{V}$	$\{X_1, X_2, X_3\}$: the variable nodes.
$\mathcal{F}$	$\{f_{12}, f_{23}\}$: the local functions, drawn as function nodes.
$\mathcal{E}$	$\{(X_1, f_{12}), (X_2, f_{12}), (X_2, f_{23}), (X_3, f_{23})\}$: the edges.

Every edge joins one variable node to one function node. The edge (X_i, f) exists exactly when $i \in \mathcal{I}_f$.

Ising model: neighboring spins interact

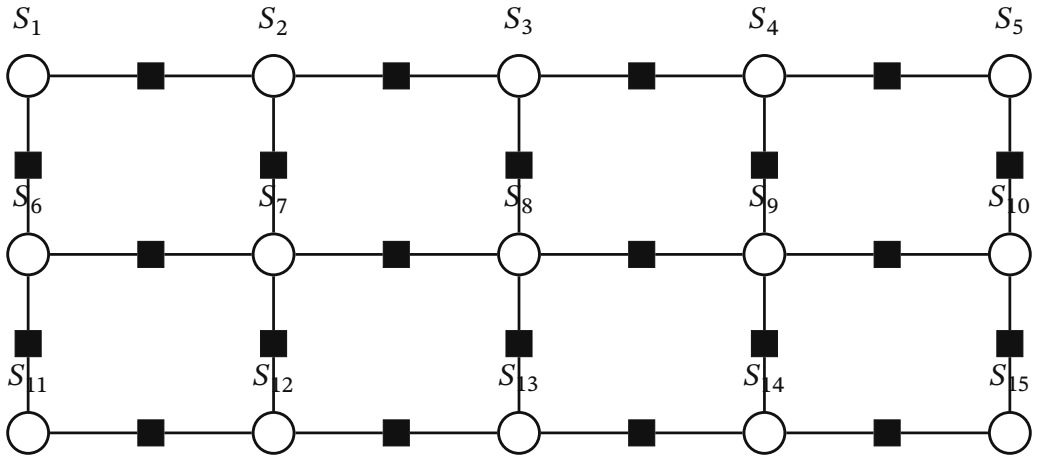
In a grid Ising model, $S_i \in \{-1, +1\}$ is the spin at site i. A neighboring pair i, j has local function

$$f_{ij}(s_i, s_j) = \exp(\beta J_{ij} s_i s_j),$$

where $\beta \geq 0$ is the inverse-temperature parameter and J_{ij} is the interaction strength.

One local function compares two neighboring spins. A large grid repeats this same relation across many sites.

Ising model: a regular grid topology



Shown: 15 spin variables and 22 neighboring-pair local functions.

The grid topology is regular, but the complete joint table still has $2^{15} = 32{,}768$ entries.

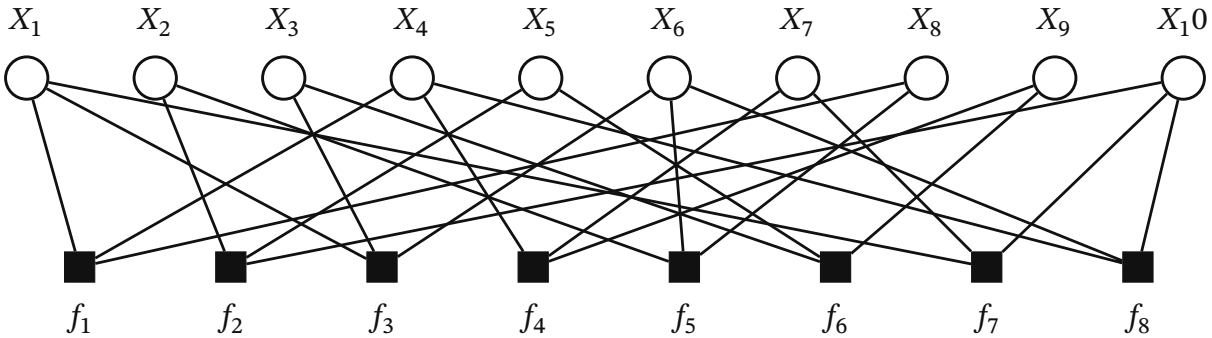
3-SAT: a clause becomes a function

In 3-SAT, $X_i \in \{0, 1\}$ is a Boolean variable. Each clause r defines a local function

$$f_r(x_{i_r}, x_{j_r}, x_{k_r}) = \begin{cases} 1, & \text{the clause is satisfied,} \\ 0, & \text{otherwise.} \end{cases}$$

In this encoding, $\prod_{r=1}^{8} f_r = 1$ exactly when **all eight clauses are satisfied**; otherwise the product is 0.

3-SAT: an irregular graph

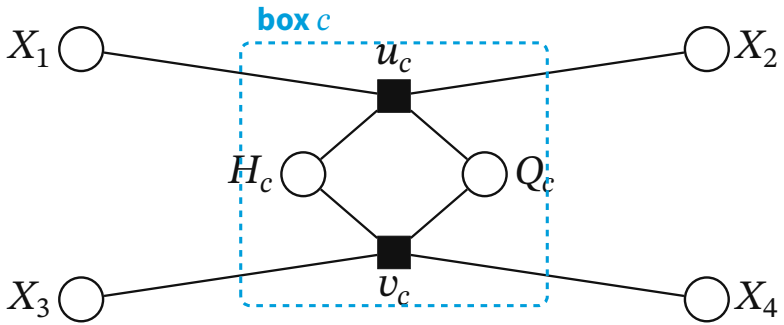


each function node connects to the three variables in one clause

Shown: 10 Boolean variables and 8 three-variable clauses.

Why the graph helps: it makes each clause's three inputs visible even when the pattern is large and irregular.

One box contains a cyclic subgraph



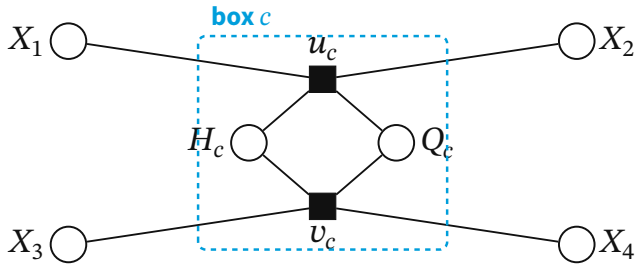
Cyclic subgraph: $H_c - u_c - Q_c - v_c - H_c$ is a closed route.

Internal variables: H_c and Q_c occur only inside the dashed box.

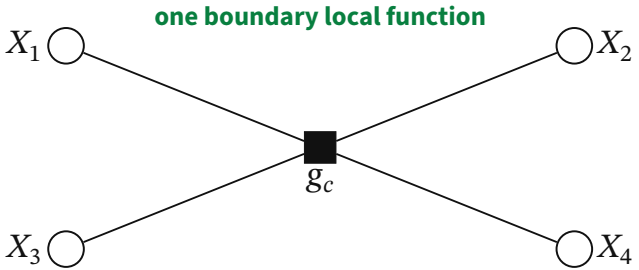
Boundary variables: X_1, X_2, X_3, X_4 connect the box to the rest of the graph.

Closing turns a subgraph into a function

Before: cyclic subgraph



After: one local function



Let $x_{\partial c} = (x_1, x_2, x_3, x_4)$ be the boundary assignment. **Your turn**

Which variables disappear inside the box? Which variables must remain as the inputs of g_c? Write the required sum–product expression.

Write before discussion.

Closing performs a reusable local calculation

1. **Fix** one assignment of the boundary variables.
2. **Sum out** every assignment of the internal variables H_c and Q_c.
3. **Store** the resulting value in g_c. Repeat for each boundary assignment.
4. **Replace** the original cyclic subgraph by g_c.

Why this can save work: later calculations reuse g_c instead of enumerating H_c and Q_c again.

When it helps: the boundary table must be much smaller than the eliminated internal calculation.

A small boundary makes closing useful

Suppose all six variables in the box are binary.

Build g_c once

4 boundary variables give $2^4 = 16$ boundary assignments.

For each one, sum over $2^2 = 4$ internal assignments.

Reuse g_c later

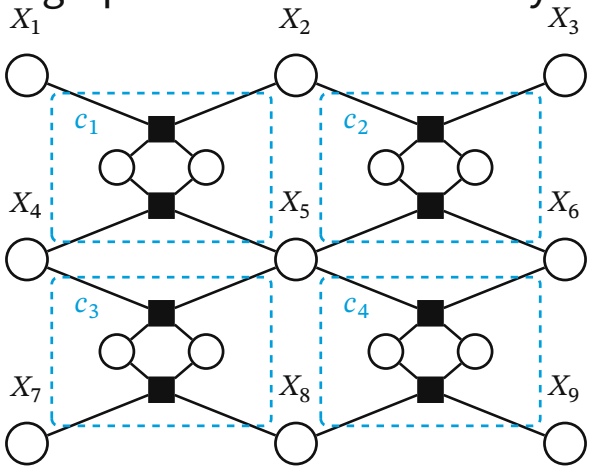
The new table stores 16 values.

The rest of the calculation reads a stored value instead of summing over H_c and Q_c again.

Closing does not remove the first calculation. It moves that calculation into a reusable table; the saving grows with reuse and shrinks with boundary size.

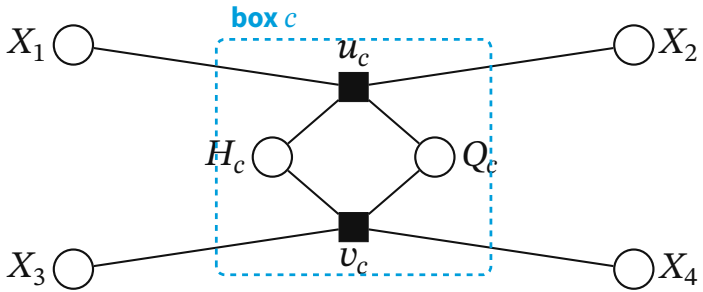
Four cyclic boxes form a grid

Four copies of the same cyclic subgraph share nine boundary variables.



Open circles: variables. **Filled squares:** local functions. **Dashed box:** one cyclic subgraph to close.

The new function has a precise domain



Boundary indices

$\partial c = \{1, 2, 3, 4\}.$

Possible boundary assignments

$\mathcal{X}_{\partial c} = \prod_{i \in \partial c} \mathcal{X}_i.$

New local function

$g_c : \mathcal{X}_{\partial c} \to \mathbb{R}_{\geq 0}.$

Output: one nonnegative value for each boundary assignment, equal to the total contribution of all internal states.

Compute the boundary function

Let $\mathcal{H}_c$ and $\mathcal{Q}_c$ be the state sets of H_c and Q_c. For each $x_{\partial c} = (x_1, x_2, x_3, x_4) \in \mathcal{X}_{\partial c}$, define **Your turn**

First multiply u_c and v_c for one fixed internal assignment. Then sum over every $h_c \in \mathcal{H}_c$ and $q_c \in \mathcal{Q}_c$.

Write before discussion.

Your turn

Mark the combine operation and the two aggregation operations in your expression.

Before and after closing the boxes



Each square g_c on the right replaces one cyclic subgraph on the left and connects to the same boundary variables.

Closing preserves the exterior function

Fix the boundary values $x_{\partial c}$. **Your turn**

State the equality that closing must preserve. What must an outside calculation receive before and after closing?

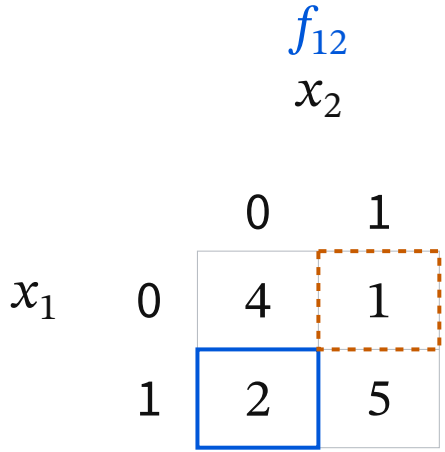
Removed from the graph: H_c, Q_c, u_c, and v_c.

Retained by the graph: the boundary variables and the new local function g_c.

Write before discussion.

The function states the lookup order

Worked example



Written order:
$f_{12}(x_1, x_2)$.

First input:
x_1 selects the row.

Second input:
x_2 selects the column.

Reversing inputs changes the question

The requested pair is $(x_1, x_2) = (1, 0)$.

Use the written order

Row $x_1 = 1$, column $x_2 = 0$:

$$f_{12}(1, 0) = 2.$$

Do not reverse the inputs

Row 0, column 1 instead gives

$$f_{12}(0, 1) = 1.$$

The second value belongs to a different machine-state pair. It is an indexing error, not another answer to the same lookup.

A graph does not specify the numbers

A complete numerical model must also state:

Variables	Meanings and domains.
Local functions	Arguments, values, and argument order.
Combination	The product that defines the global function g.
Task	The observation $X_3 = 1$ and the requested distribution of X_2.

Checkpoint: from weights to a graph

Local functions assign weights to configurations of the variables they use.

Declared operations combine local values and aggregate alternative states. This week uses multiplication and addition.

Sums and normalization produce the requested distribution.

The factor graph records which variables enter each local function.

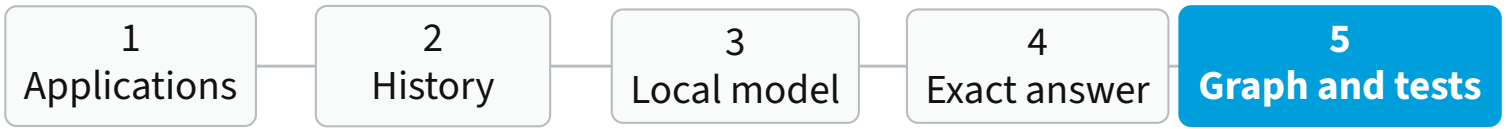
Next, test whether the model and its implementation support the reported answer.

Test the result before trusting it

Reference: compare the code with exact enumeration.

Diagnostic: change f_{12} and check that the code responds.

Boundary: detect an observation with zero model probability.



Current task: Test the result before trusting it.

Audit a coding-agent shortcut

A coding agent proposes

```
scores = f23[:, observed_x3]
return scores / sum(scores)
```

It never reads **f12**.

Your turn

Which model relationship does the shortcut discard? Why might that matter?

Audit a coding-agent shortcut

A coding agent proposes

```
scores = f23[:, observed_x3]
return scores / sum(scores)
```

It never reads **f12**.

Your turn

Which model relationship does the shortcut discard? Why might that matter?

Hint: List every array read by the two code lines, then compare the list with the model.

The first function gives equal column sums

Sum $f_{12}(x_1, x_2)$ over the two possible values of x_1 for each fixed value of x_2.

$x_2 = 0$	$x_2 = 1$
f_{12} column $x_2 = 0$: $4,\ 2$	f_{12} column $x_2 = 1$: $1,\ 5$
$\downarrow$	$\downarrow$
$4 + 2 = ____$	$1 + 5 = ____$

Your turn

Compute both column sums. Do they contribute the same multiplier?

A common multiplier disappears

The observed column of f_{23} is $(1, 4)$. Combine it with the two column sums from f_{12}.

Your turn

Write the two compatible weights. Which multiplier is common to both, and what happens to it during normalization?

Write before discussion.

Change f_{12} to test whether code uses it

Change only $f_{12}(1, 0)$ from 2 to 8.

$x_2 = 0$	$x_2 = 1$
f_{12} column $x_2 = 0$: 4, 8	f_{12} column $x_2 = 1$: 1, 5
$\downarrow$	$\downarrow$
$4 + 8 =$ ____	$1 + 5 =$ ____

Before running code: predict whether Machine 2's alert probability rises, falls, or stays unchanged.

Correct code must respond because f_{12} is part of the declared model. Code that ignores f_{12} will return the same answer.

Recalculate after the change

The observation remains $X_3 = 1$. The two column sums of the changed f_{12} are 12 and 6.

Your turn

Compute the exact probabilities. Does the shortcut respond to the changed local function?

Recalculate after the change

The observation remains $X_3 = 1$. The two column sums of the changed f_{12} are 12 and 6.

Your turn

Compute the exact probabilities. Does the shortcut respond to the changed local function?

Hint: Pair the column sums 12 and 6 with the observed-column values 1 and 4.

One controlled change exposes the bug

Your turn

Sketch the alert probability before and after the local-function change for exact enumeration and the shortcut.

Write before discussion.

Label both methods and both model settings.

A zero column makes one alert state impossible

Worked example

Suppose the model uses the following local-function table:

	$x_3 = 0$	$x_3 = 1$
$x_2 = 0$	3	**0**
$x_2 = 1$	1	**0**

The entire column for $x_3 = 1$ is zero. Therefore, for every x_1, x_2,

$$g(x_1, x_2, 1) = f_{12}(x_1, x_2)\, \underbrace{f_{23}(x_2, 1)}_{0} = 0.$$

The model assigns total probability zero to the event $X_3 = 1$.

The observation can contradict the model

Model statement: $P(X_3 = 1) = 0$; Machine 3 can never report an alert.

Observed report: $X_3 = 1$; Machine 3 does report an alert.

The conditional calculation would divide by

$$P(X_3 = 1) = 0,$$

so $p(X_2 \mid X_3 = 1)$ is **undefined**.

Correct response: report that the observation is incompatible with the model; then inspect the model table or the observation. Do not invent $(1/2, 1/2)$.

Enumeration cannot scale far

Let n be the number of binary variables. The number of complete assignments is 2^n.

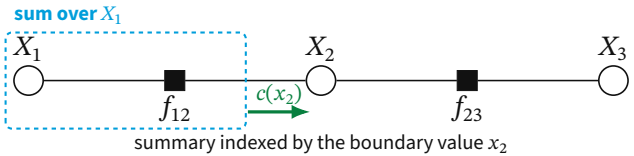
Number of variables n	3	10	20
Number of assignments 2^n	8	1,024	1,048,576

Problem: enumeration repeats the same local calculations.

Week 2 setting: a **tree** is a factor graph with no closed route along its edges.

Week 2 idea: summarize one side of the tree once, then reuse that summary.

One local sum can be reused



For each $j \in \{0, 1\}$, sum the boxed subgraph over the unobserved variable X_1:

$$c(j) = \sum_{x_1 \in \{0,1\}} f_{12}(x_1, j).$$

Here, $c(0) = 4 + 2 = 6$ and $c(1) = 1 + 5 = 6$. Reuse these two values with f_{23} instead of summing over X_1 again.

In Week 2, this reusable function of the boundary value x_2 is called a **message**.

What Week 1 establishes

1. Specify. Define every variable, possible value, local function, and operation.

2. Calculate. Retain states consistent with the observation, add their weights by the requested variable, and normalize.

3. Verify. Compare with exact enumeration; then change f_{12} and confirm that correct code responds.

4. Report limits. If the model assigns probability zero to the observation, the requested conditional probability is undefined.

Your turn

State the original-model result in one sentence.

Prepare for the laboratory and Week 2

Laboratory: calculate the probability by hand, compare the program with that result, and test a changed local function.

Week 2: on a factor graph with no cycles, each part sends a short numerical summary to its neighbor. Reuse replaces repeated enumeration.

Your turn

Predict the Week 2 result: when might reusable local summaries still recover exact marginal probabilities? State a graph condition and one reason.

Core references for graphical models

Koller and Friedman (2009), *Probabilistic Graphical Models: Principles and Techniques*. MIT Press. A broad textbook on representation, inference, and learning.

Kschischang, Frey, and Loeliger (2001), "Factor Graphs and the Sum-Product Algorithm." *IEEE Transactions on Information Theory*, 47(2), 498–519. The primary factor-graph and message-passing reference.

Begin with the textbook for context; use the paper when studying the precise factor-graph notation and the sum-product algorithm.

References for box operations

Loeliger (2004), “An Introduction to Factor Graphs.” *IEEE Signal Processing Magazine*, 21(1), 28–41. An accessible account of factor graphs and summary propagation across applications.

Forney and Vontobel (2011), “Partition Functions of Normal Factor Graphs.” A primary reference for sum-of-products representations and graph transformations that preserve an exterior function.

These papers use normal or Forney-style factor graphs. This lecture states the box operations in the variable-node and function-node notation introduced above.

References for later inference topics

Wainwright and Jordan (2008), “Graphical Models, Exponential Families, and Variational Inference.” *Foundations and Trends in Machine Learning*, 1(1–2), 1–305. The main advanced reference for information, variational methods, and approximation.

Pearl (1988), *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann. A classic account of graphical representation and probabilistic inference.

These references support the course sequence; weekly notes identify the relevant sections before each topic.